Agent harness series · Day 2 of 2

# Deep Agents

A Python agent harness built on LangGraph

Day 1 recap

# Framework, runtime and harness

### Framework

LangChain: the building blocks, such as models, tools and create_agent.

### Runtime

LangGraph: durable execution, state, checkpoints, streaming and interrupts.

### Harness

Deep Agents: the loop with file tools, subagents, summarization and approval built in.

Day 1: Mastra gave us agents plus workflows that we wired step by step. Today: a harness where the agent plans and delegates on its own.

Positioning

# Deep Agents at a glance, and when to use it

Built by LangChain. Python-first, with a JavaScript version (deepagentsjs). One call, create_deep_agent, returns a ready agent with file tools, subagents and context management, running on LangGraph.

## Choose Deep Agents when

- Your team writes Python
- Tasks are long, multi-step or open-ended
- You already use LangGraph or LangSmith

## Look elsewhere when

- You need a fixed, typed pipeline
- Your stack is TypeScript-first
- A single tool call answers the question

Setup and hello agent

# Lab 1: from install to first answer

1. **Check the pre-setup**
   Run python --version and activate a virtual environment.

2. **Install the package**
   Run pip install deepagents langchain-openai, or open the starter repo.

3. **Add your keys**
   Put the LiteLLM URL and key in .env, plus LANGSMITH_API_KEY and LANGSMITH_TRACING=true.

4. **Run it and open the trace**
   Run python hello_agent.py, then open the run in LangSmith.

Setup and hello agent

# Your first deep agent

hello_agent.py

```python
from deepagents import create_deep_agent
from model import llm  # LiteLLM url, key and model alias
agent = create_deep_agent(
    model=llm,
    system_prompt="You are a friendly assistant. Keep answers short.",
)
result = agent.invoke(
    {"messages": [{"role": "user", "content": "What is an agent harness?"}]}
)
print(result["messages"][-1].content)
```

The result is LangGraph state: a list of messages. The last message is the answer.

Core concepts and architecture

# How the pieces fit

### Input

Messages plus a thread_id

### Deep agent

create_deep_agent: the model loop plus a middleware stack

#### Tools

Your tools plus the built-in file tools

#### Subagents

The task tool spawns child agents

#### Backend

Where files live: state, disk or store

### Response

The final message, plus any files in state

Versus Mastra: Mastra gives you parts to assemble. Deep Agents gives you an assembled agent that you configure. State, checkpoints and interrupts come from LangGraph.

Core concepts and architecture

# The Deep Agents building blocks

| Concept | What it does | Where in the code |
|---|---|---|
| Deep agent | The loop with the built-in middleware stack | create_deep_agent(...) |
| Tools | Python functions or MCP tools the agent can call | tools=[...] |
| Virtual file system | ls, read_file, write_file, edit_file, glob, grep | Built in |
| Backends | Decide where files live | backend=StateBackend() and others |
| Subagents | Child agents with their own context, called with task | subagents=[...] |
| Planning | An opt-in to-do list tool, write_todos | TodoListMiddleware |
| Memory | AGENTS.md files loaded at startup | memory=[...] |
| Skills | Instructions loaded only when needed | skills=[...] |
| Summarization | Compresses long history automatically | Built in |
| Approval | Pauses before chosen tool calls | interrupt_on={...} |

Core concepts and architecture

# What the harness gives you for free

### File tools

ls, read_file, write_file, edit_file, delete, glob and grep, on whichever backend you choose.

### task

Spawns a subagent with a fresh context. It returns one final report to the main agent.

### write_todos

A structured to-do list with statuses. Opt-in: add TodoListMiddleware.

### execute

Runs shell commands. Only appears with a sandbox backend.

Your own tools sit beside these. The model reads every tool description when it decides what to call.

Tools

# Lab 2: build a tool

weather_tool.py

```python
from langchain.tools import tool

@tool
def get_weather(city: str) → dict:
    """Get the current temperature for a city."""
    return {"city": city, "temperature_celsius": 31}  # mock data

agent = create_deep_agent(model=llm, tools=[get_weather])
```

Name, docstring, type hints and body: the same four parts as Mastra's id, description, inputSchema and execute.

Tools

# MCP tools

mcp_agent.py

```python
from langchain.mcp import MCPAdapter

config = {"mcpServers": {
    "github": {"url": "https://…/mcp"},
}}

async with MCPAdapter(config) as adapter:
    tools = await adapter.list_tools()
    agent = create_deep_agent(model=llm, tools=tools)
```

**One config**
Several servers in one mcpServers map.

**Async session**
Tools stay live while the adapter is open.

**pip extra**
Install with langchain[mcp].

**Same contract**
The agent cannot tell MCP tools from local ones.

Memory and context

# Where a deep agent keeps context

### Thread state

Messages and files for one conversation, saved by a checkpointer.

### Virtual files

The agent writes notes and drafts to files instead of holding them in context.

### Long-term store

A /memories/ path routed to a store survives across threads.

### AGENTS.md

Instructions and preferences loaded on every run.

Summarization and offloading of large tool results run automatically, so long tasks stay inside the context window.

Memory and context

# Configuring memory

Backends, store and checkpointer, then the call

```
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.checkpoint.memory import MemorySaver
from langgraph.store.memory import InMemoryStore
agent = create_deep_agent(
    model=llm,
    backend=CompositeBackend(
        default=StateBackend(),  # per-thread scratch files
        routes={"/memories/": StoreBackend(namespace=lambda rt: ("memories",))},
    ),
    store=InMemoryStore(),  # long-term, across threads
    checkpointer=MemorySaver(),  # thread history
)
config = {"configurable": {"thread_id": "thread-1"}}  # thread = conversation
agent.invoke({"messages": [{"role": "user", "content": "Make it shorter"}]}, config)
```

Memory and context

# Lab 3: add memory to your agent

1. **Add a checkpointer**
   Pass checkpointer=MemorySaver() to create_deep_agent.

2. **Pass a thread id**
   Call the agent with config {"configurable": {"thread_id": "demo-thread"}}.

3. **Test a follow-up**
   Summarize solar energy, then say 'Make it shorter'. Change the thread id and it forgets.

4. **Try long-term memory**
   Ask it to save your preferences to /memories/prefs.md, then read them from a new thread.

Orchestration

# Mastra workflow versus deep agent

| Dimension | Day 1: Mastra workflow | Day 2: Deep agent |
|---|---|---|
| Who plans | You, in code, with .then() steps | The agent, with write_todos |
| Who does the work | Each step you define | Subagents it spawns with task |
| Context | Passed step to step by schema | Isolated per subagent; one report comes back |
| Predictability | Same steps every run | Varies by run, so set limits |
| Best for | Known, repeatable processes | Open-ended research and long tasks |

A deep agent is a LangGraph graph, so you can still fix the outer steps: use it as one node in a graph, or call it from a workflow step.

Orchestration

# Planning with write_todos

planning_agent.py

```python
from langchain.agents.middleware import TodoListMiddleware

agent = create_deep_agent(
    model=llm,
    tools=[lookup],
    middleware=[TodoListMiddleware()],  # adds write_todos
)
```

**Opt-in since v0.7**
Earlier versions turned it on by default.

**Three statuses**
pending, in_progress and completed.

**Kept in state**
Stream progress to a UI from the to-do list.

**When it helps**
Long tasks and smaller models.

Orchestration

# Delegation: subagents

subagents.py

```
research_subagent = {
    "name": "research-agent",
    "description": "Answers one research question with short notes and sources.",
    "system_prompt": "You are a careful researcher. Cite every source.",
    "tools": [lookup],
}
writer_subagent = {
    "name": "writer-agent",
    "description": "Turns research notes into a short summary.",
    "system_prompt": "Write clearly and keep it under 200 words.",
}
agent = create_deep_agent(model=llm, subagents=[research_subagent, writer_subagent])
```

The main agent calls them with the task tool. A general-purpose subagent is added too.

Orchestration

# Lab 4: the same research task, agent-driven

### 1 Plan

The agent turns the topic into two or three to-dos.

### 2 Delegate

It sends each sub-question to research-agent with task.

### 3 Write

writer-agent drafts the summary to /report.md.

Open the trace next to yesterday's workflow run: the same three steps, but this time the agent chose them.

Orchestration

# Human-in-the-loop: interrupt_on

### Pause before a tool

Map a tool name to True or to a config. The run stops before that call and waits.

### Decide and resume

Approve, edit the arguments, reject, or respond. Then resume the run with a Command.

A checkpointer is required: it saves the paused run so it can continue later.

Orchestration

# HITL on a tool: approve before it runs

save_note_agent.py

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

agent = create_deep_agent(
    model=llm,
    tools=[save_note],
    interrupt_on={"save_note": {"allowed_decisions": ["approve", "reject"]}},
    checkpointer=MemorySaver(),  # required
)
config = {"configurable": {"thread_id": "hitl-1"}}
result = agent.invoke({"messages": [{"role": "user", "content": "Save a note about solar energy"}]}, config)
print(result["__interrupt__"])  # the pending call
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

Orchestration

# Lab 5: add human approval

1. **Mark the tool**
   Add save_note to interrupt_on and pass a checkpointer.

2. **Run until it pauses**
   Invoke with a thread id and print result["__interrupt__"].

3. **Approve the call**
   Resume with Command(resume={"decisions": [{"type": "approve"}]}).

4. **Try rejecting**
   Run it again with {"type": "reject"} and compare the two traces in LangSmith.

Observability and evals

# Observability, guardrails and evals

### Traces

LangSmith shows every model call, tool call and subagent run, nested in order.

### Studio

langgraph dev opens LangGraph Studio to chat, inspect state and files, and step through interrupts.

### Guardrails

permissions= limits which paths the agent may read or write. Middleware adds checks such as PII detection.

Decide what good looks like: task success, right tool, grounded answers, cost and latency. LangSmith datasets and evaluators run these checks.

Mini project

# Research assistant

The same agent as Day 1: it researches any topic you give it and writes a short, sourced summary.

## What it does

- Takes a topic from the user
- Plans sub-questions with write_todos
- Researches each one through a subagent
- Writes a sourced summary to /report.md
- Remembers the topic for follow-ups

## Concepts you will use

- Deep agent and system prompt
- Tools
- Subagents and planning
- Checkpointer and backends
- Traces in LangSmith

Recap and limitations

# Strengths, limitations and pitfalls

## Strengths

- A ready harness in one call
- Subagents keep long tasks inside the context window
- LangGraph underneath: checkpoints, streaming, interrupts
- Swappable backends and sandboxes

## Limitations and pitfalls

- Python-first, and APIs change quickly
- Less control over the exact steps
- Subagents multiply calls and cost, so set limits
- Defaults hide in middleware, so read the trace

Comparison

# Mastra and Deep Agents, side by side

| Dimension | Mastra | Deep Agents |
|---|---|---|
| Language | TypeScript | Python (JS version available) |
| Kind | Framework with harness features | Harness on LangGraph |
| Tools | createTool with a Zod schema | Python function with a docstring |
| Memory | History, working, semantic, observational | Checkpointer, virtual files, store, AGENTS.md |
| Orchestration | Workflows you define, or a supervisor | The agent plans and delegates |
| Human approval | suspend and resume, requireApproval | interrupt_on and Command(resume) |
| Dev UI and traces | Studio, locally | LangSmith and LangGraph Studio |

Comparison

# When to use which

## Pick Mastra when

- Your product team writes TypeScript
- You want typed, repeatable workflows
- Agents live inside a web or API app

## Pick Deep Agents when

- Your team uses Python and LangGraph
- Tasks are long, open-ended or research-heavy
- You want planning, files and subagents ready-made

Discussion: which of our current projects fits each column, and why?

# Thank you

Questions and discussion

Docs: docs.langchain.com/oss/python/deepagents